A.

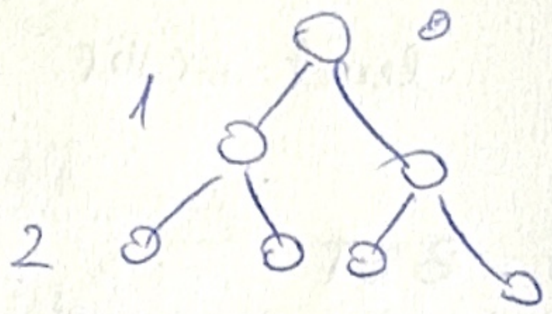
17 June 2024 → S5

1) ADT Set
search
iterator inefficient?

d) HT w s.ch. } it is better for these DS to use
e) HT w o.a. } the hash functions.

2) H = binary min. heap
>1000 ≠ int.

a) true
~~b) true~~ b false. (give ex.)
c) true



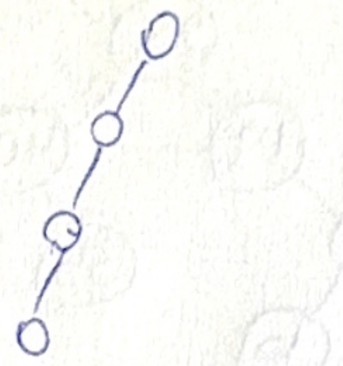
3) HT, linear probing
$h(k,i) = (h'(k) + i) \bmod m$ ← take for each value

$m = 12$

in place of xx?

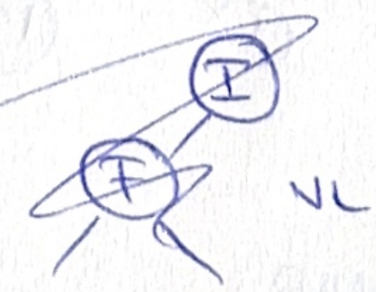
36. $h'(36) = 36 \bmod 12 = 0$
$h(36,0) = (0+0) \bmod 12 = 0$ occupied
$h(36,1) = (0+1) \bmod 12 = 1$ => f
||
xx

11 $h'(11) = 11 \bmod 12 = 11$
$h(11,0) = (11+0) \bmod 12 = 11$ occupied
$h(11,1) = 12 \bmod 12 = 0$ occupied
$h(11,2) = 13 \bmod 12 = 1$ => e
||
xx



4) Inorder: ADHXHFVLI
LRootR
Postorder: HXHDAVLFI
LRoot
Postorder = reverse of ~~P~~reorder

=> Right = ∅.



Inorder: ADHXHFVL
Postorder: HXHDAVLF

Im: ADHXH
Post: HXHDA

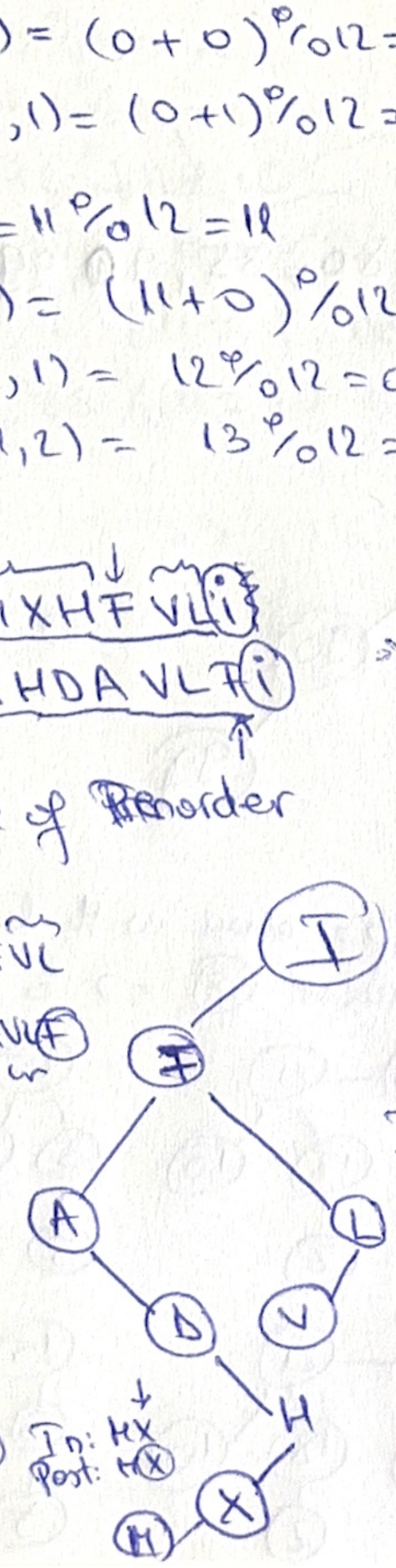
Im: DHXH
Post: HXHD

Im: HXH
Post: HXH

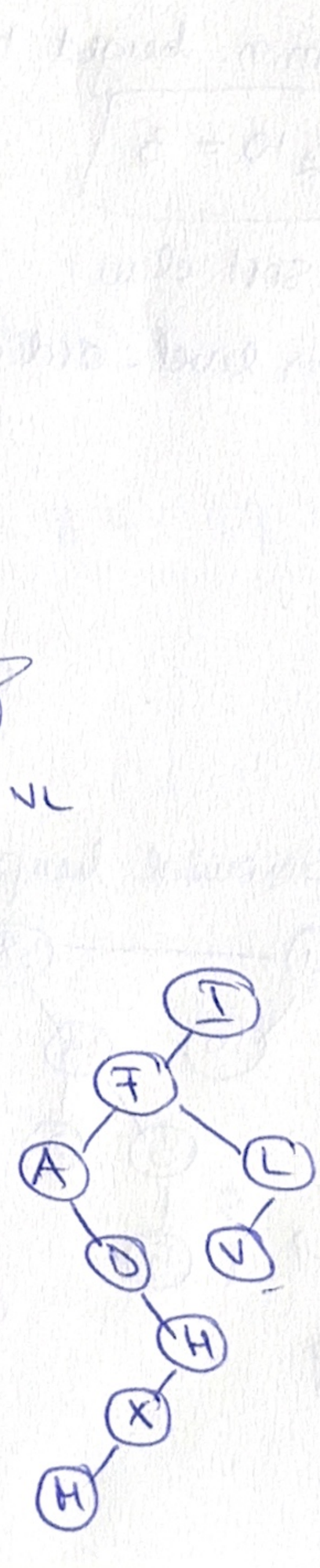
In: HX
Post: HX

Im: VL
Post: VL

Im:



=>



=> leaves:
~~H X H~~
V, H
g
c

B. 1) preorder: root - left - right

~~14 1 3~~ 14 2 1 3 11 10 7 30 40 21 ✓

postorder: left - right - root

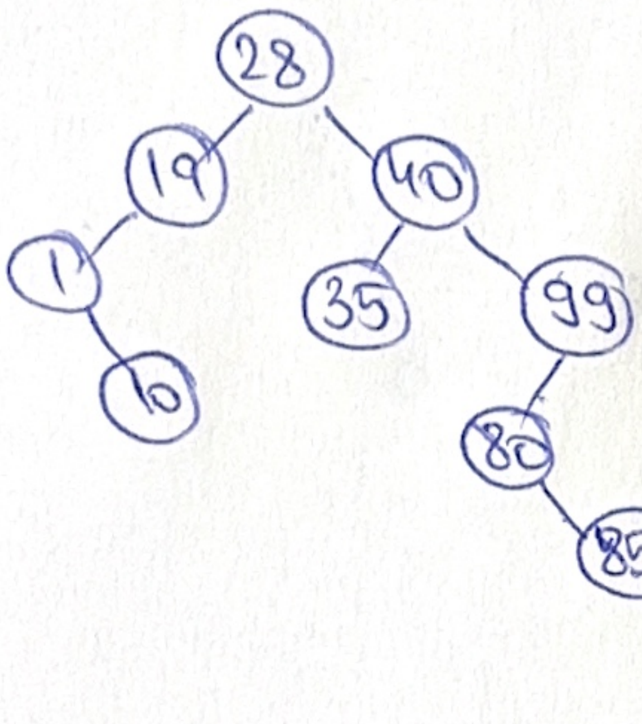
1 3 2 7 10 40 21 30 11 14

inorder: left - root - right

1 2 3 14 7 10 11 40 30 21

level order: 14 2 11 1 3 10 30 7 40 21

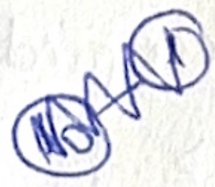
2) BST



height: 5

10 nodes

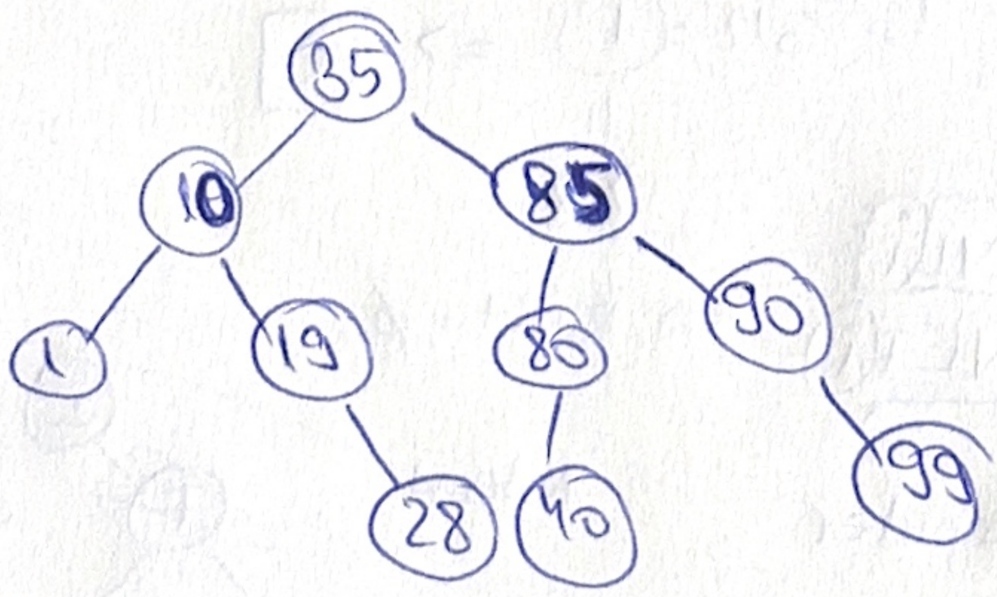
=> min. height tree:



$\log_2 10 = 3$

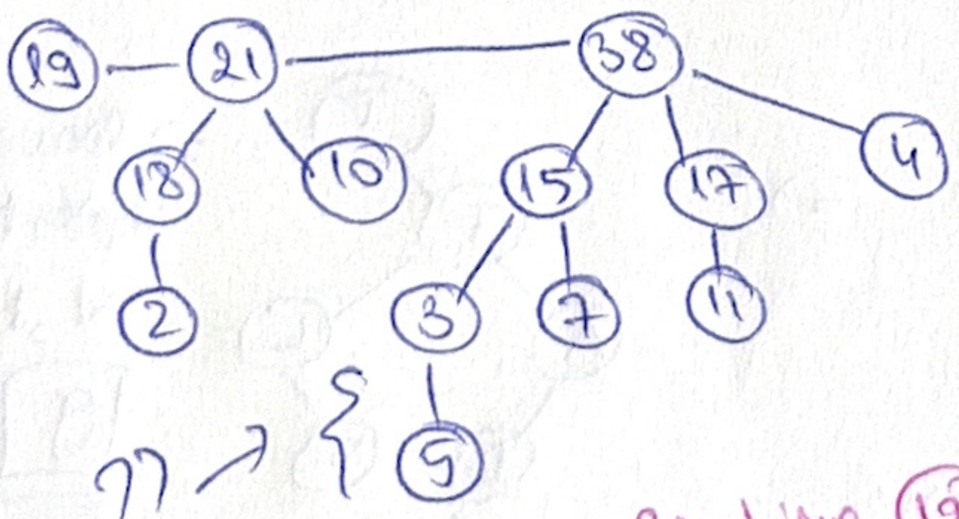
=> sort elem: 1, 10, 19, 28, 35, 40, 80, 85, 90, 99

=> level-order traversal:



=> height: 3

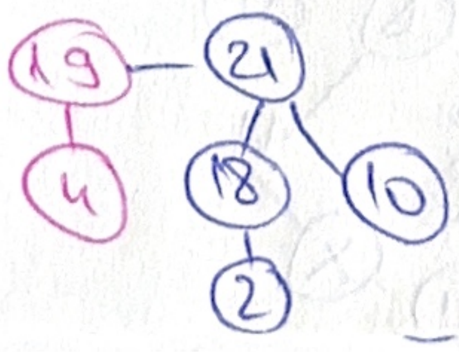
3) Binomial heap.



?? → { 5

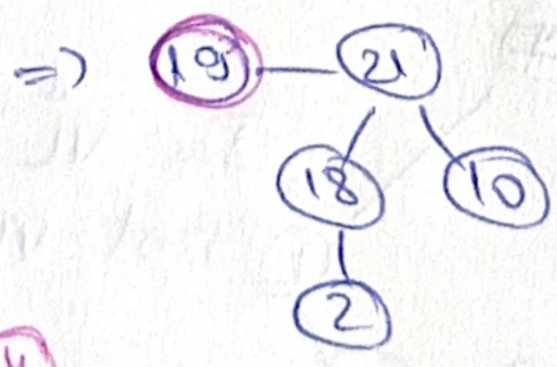
max-heap

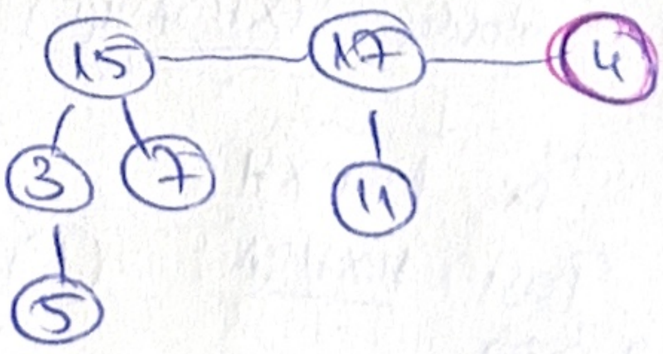
combine 19, 4 =>

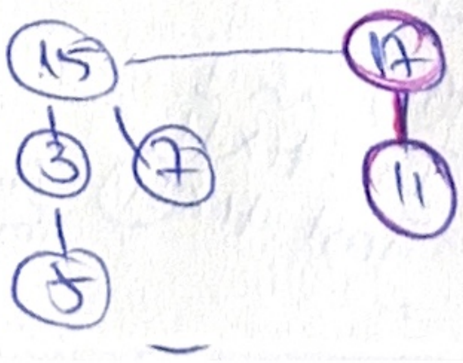


max heap => elem with highest priority: 38.

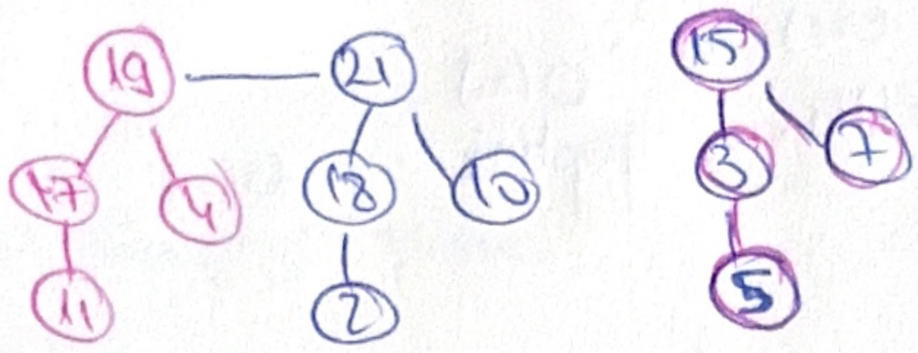
=> remove 38 => 3 trees =>



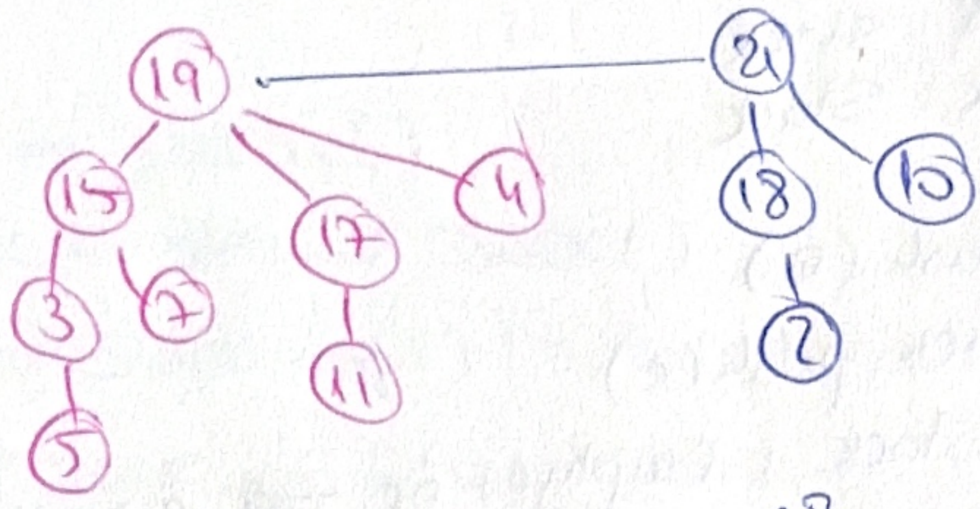




combine (19), (17) =>



combine (19) (15) =>



≈

4) m=11, HT, coalesced chaining.

98 29 84

pos	0	1	2	3	4	5	6	7	8	9	10
elms	33	12	13					7			76
next	-1			5		+6	-1	+10	6	8	9

firstEmpty : 0 1 3 5 6

h(7) = 7 % 11 = 7
h(84) = 84 % 11 = 4 (7)
h(13) = 13 % 11 = 2
h(33) = 33 % 11 = 0
h(76) = 76 % 11 = (10)
h(29) = 29 % 11 = 8 7
h(12) = 12 % 11 = 1 occupied => firstEmpty => 3
h(98) = 98 % 11 = 10 occupied => 3 => 5

remove 7 => 33 12 13 - - - - 84 98 29 76

c.

1) MaxStack → getMax, popMax $\Theta(1)$ $O(M)$ $\Theta(1)$

MaxStack:

mainStack : stack
maxStack : Stack

function push(e):
    mainStack.push(e)
    if maxStack.isEmpty() or e >= maxStack.top() then
        maxStack.push(e)
    else
        maxStack.push(maxStack.top())

function pop():
    if mainStack.isEmpty() then
        @throw ...
    mainStack.pop()
    maxStack.pop()

function top():
    if mainStack.isEmpty() then
        @throw ...
    top ← mainStack.top()

function getMax():
    if maxStack.isEmpty() then
        @throw ...
    getMax ← maxStack.top()

```
function popMax():
    if mainStack.isEmpty() then
        @throw...

    targetMax ← getMax()
    bufferStack ← new Stack()
    while mainStack.top() != targetMax do
        bufferStack.push(mainStack.top())
        pop()
    pop()
    while not bufferStack.isEmpty() do
        push(buf(S: .top())
        buf... .pop()
```